

Exercícios Práticos de Pytest

Exercício 1: Fundamentos e Asserções Básicas

Objetivo: Desenvolver e testar funções Python utilizando o Pytest e asserções nativas, compreendendo a convenção de nomenclatura e a execução básica de testes.

Desafio:

1. Crie um módulo Python (`operacoes.py`) contendo as seguintes funções:
 - `somar(a, b)`: Retorna a soma de `a` e `b`.
 - `subtrair(a, b)`: Retorna a subtração de `a` e `b`.
 - `multiplicar(a, b)`: Retorna a multiplicação de `a` e `b`.
 - `dividir(a, b)`: Retorna a divisão de `a` por `b`. Esta função deve levantar um `ValueError` se `b` for zero.
2. Crie um arquivo de testes (`test_operacoes.py`) e escreva funções de teste para cada uma das operações, seguindo as convenções de nomenclatura do Pytest (`test_` prefixo). Utilize a palavra-chave `assert` do Python para verificar os resultados das operações.
3. Para a função `dividir`, crie um teste que tente dividir por zero. Utilize `pytest.raises(ValueError)` para verificar se a exceção de divisão por zero é levantada corretamente.
4. Execute os testes usando `pytest -v` e analise a saída.

Exercício 2: Tratamento de Exceções, Números Flutuantes e Cobertura de Testes

Objetivo: Aprimorar a robustez dos testes lidando com exceções e imprecisões de ponto flutuante, e analisar a cobertura do código para identificar áreas não testadas.

Desafio:

1. Refinamento do Teste de Exceção: Adicione também um teste para uma nova função `raiz_quadrada(numero)` que você criará em `operacoes.py`, a qual deve levantar um `ValueError` se o `numero` for negativo. Teste essa exceção com `pytest.raises`.
2. Testes com Números Flutuantes: Adicione uma função `calcular_media(lista_numeros)` em `operacoes.py` que retorne a média de uma lista de números (podendo ser flutuantes). Escreva testes para essa função em `test_operacoes.py`, incluindo cenários com números de ponto

flutuante, e utilize `pytest.approx` para comparar os resultados, evitando problemas de precisão.

3. Análise de Cobertura de Código:

- Instale `pytest-cov` (`pip install pytest-cov`).
- Execute os testes para obter a cobertura de linha do seu módulo `operacoes.py`.
- Modifique uma das suas funções em `operacoes.py` para incluir uma lógica condicional (que retorne `True` ou `False`). Escreva testes que cubram ambos os caminhos (verdadeiro e falso) dessa condição.
- Execute os testes novamente, desta vez incluindo a cobertura de desvio (`--cov-branch`). Analise a saída no terminal.
- Gere um relatório HTML de cobertura e explore o arquivo `htmlcov/index.html` para visualizar quais linhas e branches do seu código foram cobertos pelos testes.

Exercício Prático 3: Integração, Lógica Complexa e Refatoração

Objetivo: Aplicar os conceitos de Pytest em um cenário que exige lógica condicional aninhada, tratamento de erros e análise crítica de cobertura de desvio.

Desafio:

1. Desenvolvimento do Módulo de Negócio (`ecommerce.py`): Crie um módulo que calcule o preço final de um produto em um e-commerce, considerando as seguintes regras:
 - `calcular_preco_final(preco_base, cupom=None, frete_gratis=False)`
 - Se o `preco_base` for negativo ou zero, deve levantar um `ValueError`.
 - Se houver um cupom válido (ex: "PROM010" dá 10% de desconto, "PROM020" dá 20%), aplique o desconto.
 - Se o cupom for inválido (qualquer outra string), não aplique desconto.
 - O frete custa R\$ 20,00. O frete será grátis (R\$ 0,00) se: o parâmetro `frete_gratis` for `True` ou se o valor do produto após o desconto for superior a R\$ 500,00.
2. Escrita de Testes Estratégicos (`test_ecommerce.py`):
 - Escreva testes que cubram os casos básicos (sucesso com e sem cupom).
 - Utilize `pytest.raises` para validar a entrada de preços inválidos.
 - Utilize `pytest.approx` para garantir que os cálculos de desconto (ponto flutuante) estejam corretos.

3. Desafio de Cobertura de Desvio:

- Execute o teste com a flag `--cov-branch`.
- Seu objetivo é atingir 100% de Branch Coverage. Para isso, você precisará identificar todos os caminhos possíveis da lógica de frete e cupons.
- Analise o relatório HTML para encontrar qualquer `BrPart` (desvio parcial) e crie o caso de teste que falta para cobrir esse caminho.